

UNIVERSIDADE PAULISTA – UNIP

Curso de Bacharelado em Sistemas da Informação

ATIVIDADE PRÁTICA SUPERVISIONADA — APS

Disciplina: Qualidade de Software

SmartBoard — Avaliação de Qualidade de Software

Gestão de Negócios Offshore e Onshore com IA Integrada

Integrantes do Grupo:

Thayna Ramos Fernandes

RA: F099EC4

Turma: Sistemas da Informação — Campus: Paraíso

Turno: Noturno

São Paulo — 2026

SUMÁRIO

1	OBJETIVO DO TRABALHO	3
2	INTRODUÇÃO	4
3	CONCEITOS GERAIS	6
3.1	Qualidade de Software e Processos de Qualidade	6
3.2	Normas ISO 9126 e ISO 14598	8
3.3	Métricas e Medição	10
3.4	Sustentabilidade no Desenvolvimento de Software	11
4	DOCUMENTO DE QUALIDADE — SmartBoard	13
4.1	Descrição do Produto e Requisitos Analisados	13
4.2	Características e Subcaracterísticas	15
4.3	Métricas e Metodologia	20
4.4	Plano de Teste	24
5	CONCLUSÃO	29
6	REFERÊNCIAS BIBLIOGRÁFICAS	31
7	FICHA DE ATIVIDADES PRÁTICAS SUPERVISIONADAS	33

1 OBJETIVO DO TRABALHO

O presente trabalho tem como objetivo central aplicar, de forma prática e sistematizada, os conceitos e métodos de avaliação de qualidade de software preconizados pelas normas ISO 9126 e ISO 14598 sobre o produto SmartBoard, desenvolvido como projeto de Trabalho de Conclusão de Curso (TCC) pelos integrantes deste grupo.

De modo específico, o trabalho visa:

- Compreender os fundamentos teóricos de qualidade de software, suas características e subcaracterísticas conforme a ISO 9126;
- Estudar o processo formal de avaliação de produtos de software definido pela ISO 14598, identificando suas fases e entregas;
- Analisar, com base na ISO 9126, no mínimo cinco requisitos funcionais e não funcionais do SmartBoard, descrevendo características, subcaracterísticas, métricas, baselines e critérios de aceite;
- Elaborar um Plano de Teste estruturado, cobrindo casos de teste, procedimentos e evidências esperadas para cada requisito avaliado;
- Avaliar aspectos de sustentabilidade do produto à luz da ISO 14001 (Sistemas de Gestão Ambiental) e da ISO Guide 82 (Sustentabilidade em Normas);
- Refletir sobre a interdisciplinaridade e contribuições desta atividade para a formação profissional do grupo.

O SmartBoard é um sistema web de gestão de negócios voltado para operações offshore (imóveis internacionais) e onshore (crédito imobiliário nacional), desenvolvido em Python/Flask com banco de dados Supabase (PostgreSQL), interface HTML/CSS/JavaScript responsiva, visualização kanban, exportação Excel e integração com agente de inteligência artificial via N8N.

2 INTRODUÇÃO

A qualidade de software deixou de ser um diferencial competitivo para se tornar requisito fundamental em qualquer produto digital moderno. Organizações de todo o mundo adotam modelos e normas internacionais para sistematizar a avaliação da qualidade, garantir a satisfação do usuário final e mitigar riscos associados a falhas de sistema. Nesse contexto, a família de normas ISO/IEC 9126 estabelece um modelo reconhecido globalmente para a caracterização e medição da qualidade de produtos de software.

O SmartBoard surgiu da necessidade identificada em uma empresa de gestão de investimentos imobiliários de consolidar, em um único ambiente digital, o acompanhamento de deals offshore (ativos imobiliários no exterior, com métricas como DSCR, LTC, Cap Rate de entrada e saída, Cash on Cash, Sensibilidade) e deals onshore (operações de crédito imobiliário no Brasil, com campos como VGV, Custo de Obra, Estrutura, Amortização, Taxas). Ferramentas genéricas como planilhas Excel e sistemas de CRM tradicionais mostravam-se insuficientes para as necessidades analíticas e de rastreabilidade do negócio.

A solução desenvolvida provê uma interface do tipo planilha interativa com edição inline, reordenação e redimensionamento de colunas via drag-and-drop, criação dinâmica de novas colunas com tipagem (texto, número, data, dropdown) e sincronização automática com o banco de dados Supabase. Além da visão tabular, o sistema oferece visualização kanban com movimentação de cards por status, dashboard analítico para cada módulo, exportação de dados em formato Excel formatado e um módulo de chat integrado a um agente de IA hospedado no N8N, capaz de consultar e atualizar registros em linguagem natural.

A autenticação é gerenciada via Supabase Auth, com fluxo completo de login, recuperação e redefinição de senha por e-mail, garantindo segurança de acesso. O backend Flask expõe uma API RESTful que cobre operações CRUD completas para ambos os módulos, além de endpoints para gestão dinâmica do schema da base de dados (adição, renomeação e exclusão de colunas em tempo de execução).

Do ponto de vista acadêmico, o SmartBoard reúne em um único produto uma gama de desafios técnicos e de qualidade relevantes para a formação em Sistemas de Informação: integração entre camadas (frontend, backend, banco de dados e serviços externos), gestão de estado e concorrência, usabilidade em interfaces ricas, segurança de autenticação, manutenibilidade do código e interoperabilidade com serviços de terceiros.

Este trabalho, portanto, apresenta a avaliação formal da qualidade do SmartBoard segundo a estrutura metodológica proposta pelas normas ISO 9126 e ISO 14598, contribuindo tanto para a evolução do produto quanto para a consolidação do aprendizado do grupo nas disciplinas de Qualidade de Software, Engenharia de Software e correlatas.

3 CONCEITOS GERAIS

3.1 Qualidade de Software e Processos de Qualidade

Qualidade de software pode ser definida como o grau em que um produto de software satisfaz requisitos explicitados e implícitos quando utilizado sob condições específicas. Segundo Pressman (2021), qualidade engloba conformidade com requisitos funcionais e de desempenho explicitamente declarados, padrões de desenvolvimento documentados e características implícitas esperadas em qualquer software desenvolvido profissionalmente.

A importância da qualidade no ciclo de vida do software é amplamente reconhecida: produtos com baixa qualidade geram custos elevados de manutenção corretiva, perda de usuários, riscos jurídicos e impacto negativo na imagem organizacional. Estima-se que corrigir defeitos após a entrega custa até cem vezes mais do que identificá-los na fase de requisitos (Boehm, 1981, atualizado por NIST, 2002).

Os processos de qualidade abrangem:

- **Garantia da Qualidade (SQA — Software Quality Assurance):** conjunto de atividades sistematizadas que fornecem confiança de que o software está sendo desenvolvido com os padrões estabelecidos. Inclui auditorias de processo, revisões técnicas e acompanhamento de métricas ao longo do ciclo de vida.
- **Controle de Qualidade (QC — Quality Control):** atividades operacionais focadas em verificar se os artefatos entregues estão em conformidade com os requisitos. Inclui inspeção de código, testes unitários, de integração e de sistema.
- **Gestão de Configuração:** controle de versões e rastreabilidade de mudanças, garantindo que as versões entregues aos usuários sejam as versões corretas e testadas.
- **Revisões Técnicas Formais (RTF):** inspeções e walkthroughs que permitem detectar defeitos precocemente, antes das fases de teste formal.

A relação entre processo de qualidade e qualidade do produto é direta: processos maduros, definidos e seguidos sistematicamente resultam em produtos com menos defeitos, maior confiabilidade e melhor aderência aos requisitos. O Modelo de Maturidade de Capacidade Integrado (CMMI) e o próprio ISO/IEC 15504 (SPICE) formalizam essa relação ao avaliar a capacidade dos processos como indicador preditivo da qualidade do produto final.

No contexto do SmartBoard, a qualidade do processo se manifesta na separação clara de responsabilidades (backend Flask, banco Supabase, frontend HTML/JS), no uso de variáveis de ambiente para configuração (evitando hardcode de credenciais), no controle de versão via Git e na documentação do projeto no README.

3.2 Normas de Qualidade: ISO 9126 e ISO 14598

A norma ISO/IEC 9126, publicada originalmente em 1991 e revisada em 2001, foi desenvolvida para padronizar a avaliação da qualidade de software por meio de um

modelo estruturado em características e subcaracterísticas. Ela é composta por quatro partes:

1. ISO/IEC 9126-1: Modelo de qualidade — define seis características principais e suas subcaracterísticas;
2. ISO/IEC 9126-2: Métricas externas — métricas aplicáveis ao comportamento do software em execução;
3. ISO/IEC 9126-3: Métricas internas — métricas aplicáveis aos artefatos de software (código, documentação) sem executá-lo;
4. ISO/IEC 9126-4: Métricas de qualidade em uso — avaliação do produto no contexto real de utilização pelo usuário final.

As seis características definidas pelo modelo são:

- Funcionalidade: capacidade do software de prover funções que atendam às necessidades declaradas e implícitas (subcaracterísticas: adequação, acurácia, interoperabilidade, conformidade, segurança de acesso);
- Confiabilidade: capacidade de manter o desempenho sob condições e por um período definido (subcaracterísticas: maturidade, tolerância a falhas, recuperabilidade);
- Usabilidade: facilidade de uso pelo usuário-alvo (subcaracterísticas: inteligibilidade, apreensibilidade, operacionalidade, atratividade);
- Eficiência: relação entre o nível de desempenho e os recursos consumidos (subcaracterísticas: comportamento em relação ao tempo, utilização de recursos);
- Manutenibilidade: facilidade de modificar o software para correções, melhorias ou adaptação (subcaracterísticas: analisabilidade, modificabilidade, estabilidade, testabilidade);
- Portabilidade: capacidade de ser transferido entre diferentes ambientes (subcaracterísticas: adaptabilidade, capacidade para instalação, coexistência, capacidade para substituir).

A ISO/IEC 9126 foi sucedida pela família ISO/IEC 25000 (SQuaRE — System and Software Quality Requirements and Evaluation), que expande e atualiza o modelo, mas a ISO 9126 ainda é amplamente utilizada academicamente e como referência nos currículos universitários brasileiros.

A norma ISO/IEC 14598 complementa a ISO 9126 ao definir um processo formal de avaliação de produtos de software. Ela é composta por seis partes, sendo a mais relevante a ISO/IEC 14598-1 (Visão Geral) e a ISO/IEC 14598-5 (Processo para avaliadores). O processo de avaliação previsto pela norma segue quatro etapas:

5. Estabelecimento dos requisitos de avaliação: definição do propósito, escopo e requisitos de qualidade que se deseja medir;
6. Especificação da avaliação: seleção de métricas, definição de níveis de pontuação e critérios de aceite;
7. Projeto da avaliação: planejamento detalhado das atividades de medição e coleta de dados;

8. Execução da avaliação: aplicação das métricas, interpretação dos resultados e elaboração do relatório de avaliação.

A relação entre ISO 9126 e ISO 14598 é de complementaridade: a primeira define O QUE medir (características e métricas de qualidade), enquanto a segunda define COMO medir (processo de avaliação). Juntas, formam o arcabouço metodológico utilizado neste trabalho para avaliar o SmartBoard.

3.3 Conceitos de Métricas e Medição

Uma métrica de software é uma medida quantitativa do grau em que um sistema, componente ou processo possui um determinado atributo (IEEE, 1990). Métricas permitem transformar percepções subjetivas sobre qualidade em dados objetivos, comparáveis e rastreáveis ao longo do tempo.

Existem três categorias principais de métricas segundo a ISO 9126:

- Métricas internas: aplicadas aos produtos intermediários (código-fonte, documentação, modelos). Exemplos: cobertura de código por testes, complexidade ciclomática, número de linhas de código (LOC);
- Métricas externas: observadas no comportamento do software em execução. Exemplos: tempo de resposta de uma requisição HTTP, taxa de erros em produção, disponibilidade do serviço;
- Métricas de qualidade em uso: avaliadas na perspectiva do usuário real no contexto de uso. Exemplos: taxa de conclusão de tarefas, número de erros cometidos pelo usuário, satisfação medida por questionários.

O processo de medição, segundo a ISO/IEC 15939 (Medição de Software), envolve: identificação do objeto de medição, seleção do atributo a medir, definição da forma de coleta, estabelecimento de baseline (valor de referência ou histórico) e definição do critério de aceite. O baseline pode ser derivado de normas da indústria, benchmarks, medições históricas do próprio produto ou metas definidas pelos stakeholders.

3.4 Sustentabilidade e Aplicação no Desenvolvimento de Software

A sustentabilidade no contexto do desenvolvimento de software é um tema crescente, impulsionado pela crescente consciência ambiental e pela pressão de regulações internacionais. A ISO 14001:2015 define os requisitos de um Sistema de Gestão Ambiental (SGA), estabelecendo um ciclo PDCA (Plan-Do-Check-Act) para identificar, controlar e reduzir os impactos ambientais de uma organização.

Aplicada ao desenvolvimento de software, a ISO 14001 orienta as equipes a considerar:

- Consumo de energia dos ambientes de computação em nuvem utilizados (datacenters, serviços SaaS);
- Eficiência dos algoritmos e do código, que impacta diretamente o consumo de energia dos servidores e dos dispositivos dos usuários;
- Ciclo de vida do hardware necessário para executar o software, incentivando o uso de infraestruturas compartilhadas (cloud) em detrimento de servidores físicos dedicados;

- Descarte adequado de dados (LGPD e políticas de retenção) para reduzir o volume de dados armazenados desnecessariamente.

A ISO Guide 82 (Guidelines for addressing sustainability in standards) complementa a ISO 14001 ao orientar os criadores de normas a incorporar considerações de sustentabilidade — ambiental, econômica e social — em todas as etapas de desenvolvimento de padrões. Para produtos de software, a Guide 82 sugere avaliar:

- Impacto ambiental ao longo do ciclo de vida do produto (Green Software Engineering);
- Sustentabilidade econômica: custo total de propriedade (TCO), incluindo licenças, infraestrutura, manutenção e treinamento;
- Sustentabilidade social: acessibilidade, inclusão digital e impacto nas partes interessadas.

No contexto do SmartBoard, a escolha do Supabase (infraestrutura em nuvem com datacenters certificados em padrões de eficiência energética) e do N8N (plataforma de automação que pode ser executada localmente ou em cloud compartilhada, reduzindo a necessidade de infraestrutura dedicada) demonstra uma orientação natural para práticas de desenvolvimento sustentável. O uso de Python — linguagem interpretada com amplo suporte a boas práticas de eficiência algorítmica — e a arquitetura serverless do Supabase, que escala sob demanda sem manter servidores ociosos, contribuem para a redução do consumo energético comparado a arquiteturas monolíticas tradicionais.

Do ponto de vista da sustentabilidade econômica, o SmartBoard utiliza o tier gratuito do Supabase para desenvolvimento e prototipação, reduzindo o custo de validação da solução antes de escalar para planos pagos. A arquitetura modular (Flask + Supabase + N8N) permite substituir componentes individualmente sem reescrever o sistema inteiro, reduzindo o custo de manutenção a longo prazo.

4 DOCUMENTO DE QUALIDADE — SMARTBOARD

4.1 Descrição do Produto e Requisitos Analisados

O SmartBoard é um sistema web de gestão de deals (negócios) desenvolvido em Python com o framework Flask, com frontend em HTML5/CSS3/JavaScript puro e banco de dados PostgreSQL gerenciado pelo serviço Supabase. O produto é acessado via navegador web em endereço local (<http://localhost:5000>) ou em ambiente de produção.

Arquitetura do sistema:

- Backend: Python 3.11 + Flask — servidor HTTP que expõe rotas de páginas (renderização de templates Jinja2) e endpoints REST para operações CRUD e gestão de schema;
- Banco de dados: Supabase (PostgreSQL) — banco relacional com dois módulos principais (deals_offshore e deals_onshore), tabela de configuração de colunas customizadas (col_config) e funções SQL auxiliares (exec_sql, get_table_cols);
- Frontend: HTML/CSS/JavaScript — interface responsiva com planilha interativa, visualização kanban, dashboards analíticos e módulo de chat;
- Autenticação: Supabase Auth — fluxo completo de login, recuperação de senha e sessão segura via Flask session;
- Integração de IA: N8N via webhook — agente conversacional que interpreta linguagem natural, consulta e atualiza dados no Supabase;
- Exportação: openpyxl — geração de planilhas Excel formatadas com cabeçalho estilizado, zebra striping e ajuste automático de largura de colunas.

Os cinco requisitos selecionados para análise de qualidade são:

ID	Requisito	Descrição	Categoria
REQ-01	Autenticação e Controle de Acesso	O sistema deve autenticar usuários via e-mail/senha e proteger todas as rotas com verificação de sessão ativa.	Funcionalidade — Segurança de Acesso
REQ-02	Edição e Persistência de Dados da Planilha	O usuário deve ser capaz de editar células com duplo clique e os dados devem ser persistidos no banco em até 3 segundos.	Funcionalidade — Acurácia; Eficiência — Comportamento Temporal
REQ-03	Adição Dinâmica de Colunas	O usuário deve poder adicionar novas colunas ao schema da tabela em tempo de execução, sem necessidade de reinicialização do servidor.	Funcionalidade — Adequação; Manutenibilidade — Modificabilidade
REQ-04	Visualização Kanban com Drag-and-Drop	O sistema deve exibir os deals em formato kanban por status e permitir mover cards entre colunas, atualizando o status no banco automaticamente.	Usabilidade — Operacionalidade; Funcionalidade — Acurácia

ID	Requisito	Descrição	Categoria
REQ-05	Integração com Agente de IA via Chat	O módulo de chat deve enviar mensagens do usuário ao agente N8N e exibir a resposta de forma inteligível, permitindo consultas e atualizações de dados em linguagem natural.	Funcionalidade — Interoperabilidade; Usabilidade — Inteligibilidade

4.2 Características e Subcaracterísticas Analisadas

4.2.1 REQ-01 — Autenticação e Controle de Acesso

Característica principal: Funcionalidade

Subcaracterística: Segurança de Acesso — capacidade de prevenir acesso não autorizado, acidental ou deliberado, a programas e dados.

Análise no SmartBoard: O sistema implementa autenticação via Supabase Auth, com login por e-mail e senha. Todas as rotas de negócio (/offshore, /onshore, /chat, /dashboard/*) são protegidas pelo decorator login_required, que verifica a existência da chave user_id na sessão Flask antes de processar qualquer requisição. O fluxo de recuperação de senha utiliza o mecanismo de e-mail do Supabase Auth com redirecionamento parametrizado, e o token de recovery é validado no lado do servidor antes de permitir a troca de senha.

Subcaracterística secundária: Conformidade — o módulo de autenticação segue padrões estabelecidos de segurança web (sessão server-side, HTTPS recomendado em produção, tokens JWT gerenciados pelo Supabase).

4.2.2 REQ-02 — Edição e Persistência de Dados da Planilha

Característica principal: Funcionalidade

Subcaracterística: Acurácia — capacidade de fornecer resultados ou efeitos corretos ou conforme acordado, com o grau de precisão necessário.

Análise no SmartBoard: A função to_db_row() realiza conversão e validação de tipos antes de persistir os dados: campos numéricos (FLOAT_FIELDS e INTEGER_FIELDS) passam por limpeza de símbolos monetários e de porcentagem, com conversão segura usando try/except. Campos não numéricos têm valores em branco convertidos para None (NULL no banco). Isso garante que os dados armazenados sejam do tipo e formato corretos.

Característica secundária: Eficiência

Subcaracterística: Comportamento Temporal — tempo de resposta e tempo de processamento sob condições estabelecidas.

Análise: As operações de atualização são feitas via requisição PUT ao endpoint /api/<table>/id com resposta JSON. A operação envolve chamada HTTP ao Supabase, que executa UPDATE no PostgreSQL e retorna o registro atualizado. O tempo total inclui latência de rede (cliente-Flask, Flask-Supabase).

4.2.3 REQ-03 — Adição Dinâmica de Colunas

Característica principal: Funcionalidade

Subcaracterística: Adequação — capacidade de prover um conjunto apropriado de funções para tarefas e objetivos do usuário especificados.

Análise no SmartBoard: O endpoint POST /api/<table>/columns executa ALTER TABLE via função RPC exec_sql no Supabase, registra a configuração da nova coluna (tipo e opções) na tabela col_config e retorna imediatamente a confirmação. Isso permite ao usuário personalizar o schema sem qualquer intervenção técnica ou reinicialização do sistema.

Característica secundária: Manutenibilidade

Subcaracterística: Modificabilidade — esforço necessário para fazer modificações especificadas.

Análise: A arquitetura dinâmica de colunas — com mapeamento label/col separado, função label_to_col() para normalização e tabela col_config como registro das customizações — demonstra design orientado à manutenibilidade: novas colunas podem ser adicionadas, renomeadas ou removidas sem alterar código-fonte.

4.2.4 REQ-04 — Visualização Kanban com Drag-and-Drop

Característica principal: Usabilidade

Subcaracterística: Operacionalidade — capacidade de ser fácil de operar e controlar.

Análise no SmartBoard: A visualização kanban agrupa os deals por status em colunas distintas. O usuário pode arrastar um card de uma coluna para outra, o que dispara uma requisição PUT atualizando o campo status no banco de dados. A operação é imediata e visível, sem necessidade de salvar manualmente ou recarregar a página.

Subcaracterística secundária: Atratividade — capacidade de ser atraente ao usuário.

Análise: A interface utiliza design system consistente (fonte DM Sans, paleta azul-marinho #0F2044 + laranja #F15A24, cards com sombra e bordas arredondadas), proporcionando uma experiência visual coerente e agradável. A alternância entre visualização tabular e kanban via toggle button com ícones SVG aumenta a expressividade da interface.

4.2.5 REQ-05 — Integração com Agente de IA via Chat

Característica principal: Funcionalidade

Subcaracterística: Interoperabilidade — capacidade de interagir com sistemas especificados.

Análise no SmartBoard: O módulo de chat envia o histórico de mensagens ao webhook N8N via requisição POST com payload JSON. O backend Flask atua como proxy, adicionando timeout de 30 segundos e tratamento de erros HTTP. A resposta do N8N é extraída do campo output, message, text ou response do JSON retornado, garantindo compatibilidade com diferentes configurações do agente.

Característica secundária: Usabilidade

Subcaracterística: Inteligibilidade — capacidade de possibilitar ao usuário entender se o software é apropriado, e como ele pode ser usado para tarefas e condições de uso particulares.

Análise: A tela de chat exibe sugestões de perguntas na tela inicial ("Analise o DSCR dos deals offshore", "Quais deals estão com maior risco?", etc.), reduzindo a curva de aprendizado e orientando o usuário sobre as capacidades do agente.

4.3 Métricas e Metodologia Adotadas

A metodologia adotada segue o processo de avaliação definido pela ISO 14598-5:

9. Estabelecimento dos requisitos de avaliação: identificação dos cinco requisitos críticos do SmartBoard;
10. Especificação da avaliação: seleção de métricas, definição de baselines e critérios de aceite;
11. Projeto da avaliação: definição dos casos de teste e procedimentos de medição;
12. Execução da avaliação: aplicação dos casos de teste, coleta de resultados e interpretação.

As tabelas a seguir detalham as métricas, baselines e critérios de aceite para cada requisito:

Requisito	Métrica	Método de Medição	Baseline	Critério de Aceite
REQ-01 (Autenticação)	Taxa de bloqueio de acesso não autorizado (%)	Tentativa de acesso direto a 10 rotas protegidas sem sessão ativa; contar redirecionamentos para /login	100% de bloqueio (OWASP Top 10)	≥ 100% de redirecionamentos corretos
REQ-01 (Autenticação)	Taxa de sucesso no login com credenciais válidas (%)	Executar 20 logins com credenciais corretas; medir taxa de resposta HTTP 200 com redirect para /	≥ 99% (OWASP)	≥ 95% de sucesso
REQ-02 (Persistência)	Acurácia da persistência de dados (%)	Editar 30 células com valores variados (texto, número, data, dropdown); comparar valor salvo no banco com valor inserido	100% de acurácia esperada	≥ 97% de acurácia nos dados persistidos
REQ-02 (Persistência)	Tempo de resposta do PUT (ms)	Medir tempo de resposta de 50 requisições PUT via DevTools Network ou	< 3.000 ms (requisito do negócio)	P95 < 3.000 ms; P50 < 1.500 ms

Requisito	Métrica	Método de Medição	Baseline	Critério de Aceite
		ferramenta de benchmark HTTP		
REQ-03 (Colunas)	Taxa de sucesso na adição de coluna (%)	Adicionar 10 colunas de tipos distintos; verificar criação no schema e registro no col_config	100% de sucesso esperado	≥ 90% de sucesso sem reinicialização do servidor
REQ-04 (Kanban)	Taxa de atualização de status via drag-and-drop (%)	Mover 20 cards entre colunas; verificar no banco se o campo status foi atualizado corretamente	100% de atualização esperada	≥ 95% de atualizações corretas no banco
REQ-05 (Chat IA)	Taxa de resposta do agente IA (%)	Enviar 20 mensagens válidas ao chat; contar respostas recebidas dentro do timeout de 30s	≥ 95% (SLA típico de serviços cloud)	≥ 90% de respostas dentro do timeout
REQ-05 (Chat IA)	Taxa de inteligibilidade das sugestões (%)	Avaliação heurística por 5 usuários: cada um avalia se as sugestões exibidas na tela inicial comunicam corretamente o que o agente pode fazer (escala 1-5)	Média ≥ 4,0 (Nielsen Heuristics)	Média ≥ 3,5 (70% de aprovação)

4.4 Plano de Teste

4.4.1 Escopo e Objetivos do Plano de Teste

O Plano de Teste cobre a verificação e validação dos cinco requisitos selecionados (REQ-01 a REQ-05) do SmartBoard, utilizando técnicas de teste de caixa preta (comportamento observável) e caixa branca (análise de código e logs internos). Os objetivos são:

- Verificar se cada requisito atende ao seu critério de aceite definido na seção 4.3;
- Identificar defeitos e não conformidades antes da entrega final do produto;
- Gerar evidências documentadas para validação acadêmica e registro de qualidade.

4.4.2 Ambiente de Teste

Os testes serão executados no seguinte ambiente:

- Sistema operacional: Windows 11 (desenvolvimento) e/ou Ubuntu 22.04 LTS (servidor);
- Navegadores: Google Chrome versão mais recente, Mozilla Firefox versão mais recente;
- Python: 3.11, com ambiente virtual venv;
- Banco de dados: Supabase (projeto de desenvolvimento, isolado do projeto de produção);
- Ferramentas de teste: DevTools (Network, Console), Postman (testes de API), pytest (testes unitários do backend), planilha de controle de casos de teste (Excel).

4.4.3 Casos de Teste — REQ-01: Autenticação e Controle de Acesso

ID CT	Pré-condição	Passos	Dados de Entrada	Resultado Esperado	Critério
CT-01-01	Usuário não autenticado	1. Acessar diretamente /offshore sem login; 2. Verificar resposta HTTP e URL	URL: http://localhost:5000/offshore	Redirecionamento para /login; HTTP 302	BLOQUEIO
CT-01-02	Usuário não autenticado	1. Acessar /api/offshore sem sessão; 2. Verificar resposta	GET /api/offshore (sem cookie de sessão)	Redirecionamento ou HTTP 401/403	BLOQUEIO
CT-01-03	Usuário cadastrado no Supabase (Auto Confirm)	1. Acessar /login; 2. Informar e-mail e senha corretos; 3. Clicar em Entrar	E-mail: usuario@teste.com; Senha: correta123	Redirecionamento para /offshore; sessão criada com user_id	ACEITE
CT-01-04	Usuário cadastrado	1. Informar e-mail correto e senha errada; 2. Clicar em Entrar	E-mail: usuario@teste.com; Senha: errada999	Mensagem de erro; nenhuma sessão criada; permanece em /login	REJEIÇÃO CORRETA
CT-01-05	Usuário autenticado	1. Clicar em Sair no painel lateral; 2. Confirmar logout; 3. Tentar	Ação de logout	Sessão destruída; redirecionamento para /login ao tentar acessar rota protegida	LOGOUT OK

ID CT	Pré-condição	Passos	Dados de Entrada	Resultado Esperado	Critério
		acessar /offshore			

4.4.4 Casos de Teste — REQ-02: Edição e Persistência de Dados

ID CT	Pré-condição	Passos	Dados de Entrada	Resultado Esperado	Critério
CT-02-01	Usuário logado; deal existente em offshore	1. Dar duplo clique em célula de texto; 2. Alterar valor; 3. Pressionar Enter; 4. Consultar banco	Campo Tarefa: novo valor de texto alfanumérico	Valor exibido na célula = valor no banco; sem truncamento	ACEITE
CT-02-02	Usuário logado; deal existente	1. Editar campo numérico DSCR; 2. Inserir "1,25"; 3. Confirmar; 4. Verificar banco	Campo DSCR: "1,25" (formato BR com vírgula)	Banco armazena 1.25 (float); conversão correta de vírgula para ponto	ACEITE
CT-02-03	Usuário logado	1. Editar campo numérico com valor inválido (texto puro); 2. Confirmar	Campo DSCR: "abc"	Campo armazenado como NULL; sem erro 500; interface notifica ou limpa o campo	ACEITE (tolerância)
CT-02-04	Usuário logado; conexão estável	1. Editar célula; 2. Medir tempo entre clique em Enter e retorno visual de confirmação	Qualquer campo; medição via DevTools Network	P95 do tempo de resposta < 3.000 ms	ACEITE

4.4.5 Casos de Teste — REQ-03: Adição Dinâmica de Colunas

ID CT	Pré-condição	Passos	Dados de Entrada	Resultado Esperado	Critério
CT-03-01	Usuário logado; tabela offshore aberta	1. Clicar no ícone de adicionar coluna; 2. Digitar nome; 3. Selecionar tipo Texto; 4. Confirmar	Nome: "Observação"; Tipo: Texto	Nova coluna visível na tabela; col_config registrada; ALTER TABLE executado no banco	ACEITE
CT-03-02	Usuário logado	1. Adicionar coluna do tipo Dropdown; 2. Informar	Nome: "Prioridade"; Tipo: Dropdown; Opções: ["Alta", "Média", "Baixa"]	Coluna exibida como dropdown; opções corretas armazenadas em	ACEITE

ID CT	Pré-condição	Passos	Dados de Entrada	Resultado Esperado	Critério
		opções; 3. Confirmar; 4. Verificar col_config		col_config.options (JSONB)	
CT-03-03	Usuário logado	1. Tentar adicionar coluna com nome já existente; 2. Confirmar	Nome: "tarefa" (coluna já existente)	Banco executa IF NOT EXISTS; nenhum erro crítico; comportamento previsível	ACEITE (idempotência)

4.4.6 Casos de Teste — REQ-04: Visualização Kanban

ID CT	Pré-condição	Passos	Dados de Entrada	Resultado Esperado	Critério
CT-04-01	Deals com status variados; usuário logado	1. Clicar em toggle Kanban; 2. Verificar agrupamento por status	Dados de deals com status: Ativo, Análise, Pausado, Concluído	Cada deal aparece na coluna correta do kanban	ACEITE
CT-04-02	Kanban visível com pelo menos 2 colunas de status	1. Arrastar card de "Ativo" para "Concluído"; 2. Verificar status no banco	Card do deal ID X; destino: coluna "Concluído"	Card removido da coluna "Ativo"; exibido em "Concluído"; banco atualizado (status = "Concluído")	ACEITE
CT-04-03	Kanban visível	1. Alternar entre visualização Tabela e Kanban; 2. Verificar consistência dos dados	Toggle entre modos	Dados idênticos em ambas as visualizações; sem perda de registros	ACEITE

4.4.7 Casos de Teste — REQ-05: Chat com Agente de IA

ID CT	Pré-condição	Passos	Dados de Entrada	Resultado Esperado	Critério
CT-05-01	Usuário logado; N8N webhook ativo	1. Acessar /chat; 2. Digitar pergunta válida; 3. Enviar; 4. Aguardar resposta	Mensagem: "Quanto deals offshore estão ativos?"	Resposta exibida dentro de 30 segundos; conteúdo relacionado à pergunta	ACEITE

ID CT	Pré-condição	Passos	Dados de Entrada	Resultado Esperado	Critério
CT-05-02	Usuário logado; N8N indisponível (simulado)	1. Enviar mensagem ao chat com N8N fora do ar; 2. Verificar tratamento de erro	Qualquer mensagem; N8N retorna HTTP 503	Interface exibe mensagem de erro amigável; nenhuma exceção não tratada; HTTP 502 retornado pelo Flask	ACEITE (resiliência)
CT-05-03	Tela de chat sem mensagens anteriores	1. Verificar se sugestões de perguntas estão visíveis; 2. Clicar em uma sugestão	Nenhuma ação prévia	4 sugestões visíveis na tela inicial; clique em sugestão preenche o campo de texto	ACEITE (UX)

5 CONCLUSÃO

A aplicação sistemática das normas ISO 9126 e ISO 14598 ao SmartBoard revelou que o produto, embora desenvolvido em contexto acadêmico-profissional com recursos limitados, apresenta um nível satisfatório de qualidade nas características avaliadas, com pontos de excelência e oportunidades claras de melhoria.

Do ponto de vista da Funcionalidade, o sistema demonstra adequação ao propósito declarado: gestão de deals imobiliários offshore e onshore, com CRUD completo, schema dinâmico, exportação de dados e integração com agente de IA. A segurança de acesso, implementada via decorator `login_required` em todas as rotas protegidas e autenticação gerenciada pelo Supabase Auth, atende plenamente ao critério de aceite de 100% de bloqueio de acessos não autorizados.

A Acurácia dos dados, garantida pela função `to_db_row()` com conversão e validação de tipos, assegura que informações financeiras críticas — como DSCR, LTC, Cap Rate e percentuais de vendas e obra — sejam armazenadas com o formato numérico correto, evitando distorções analíticas. A Eficiência Temporal, avaliada pelo tempo de resposta das requisições de atualização, depende fortemente da latência da conexão com o Supabase, o que aponta para a necessidade de monitoramento de SLA em produção.

A Usabilidade do SmartBoard é um dos seus pontos mais fortes: a interface de duplo clique para edição, o drag-and-drop no kanban, as sugestões no chat e o design system consistente (DM Sans, paleta azul-marinho/laranja) conferem uma experiência intuitiva. A Manutenibilidade é favorecida pela arquitetura modular, pelo design de colunas dinâmicas e pela separação clara de responsabilidades entre backend Flask, banco Supabase e frontend, facilitando evoluções futuras sem grandes refatorações.

As principais dificuldades encontradas na identificação de métricas foram:

- Definição de baselines para métricas de tempo de resposta em ambiente local versus produção: valores obtidos em localhost diferem significativamente dos valores em produção com conexões reais à internet;
- Quantificação objetiva de subcaracterísticas subjetivas como Atratividade e Inteligibilidade, que requerem avaliação com usuários reais e aplicação de métodos como SUS (System Usability Scale) ou avaliação heurística de Nielsen;
- Isolamento de variáveis no teste de integração com o N8N, cujo comportamento depende de serviços externos (LLM providers, banco de dados do agente), tornando difícil garantir reprodutibilidade dos testes.

Em relação à interdisciplinaridade, este trabalho integrou conteúdos de diversas disciplinas do curso de Sistemas de Informação:

- Engenharia de Software: modelagem de requisitos, arquitetura de software e gestão de configuração;
- Banco de Dados: modelagem relacional, SQL dinâmico, triggers e funções no PostgreSQL;
- Programação Web: Flask, Jinja2, HTML/CSS/JavaScript, APIs REST e consumo de webhooks;

- Segurança da Informação: autenticação, controle de sessão, proteção de endpoints e OWASP Top 10;
- Inteligência Artificial: integração com agentes LLM e automação de processos via N8N;
- Sustentabilidade: avaliação do impacto ambiental e econômico das escolhas de infraestrutura (Supabase, N8N, arquitetura cloud-first).

Quanto ao efeito na formação do grupo, a realização deste trabalho proporcionou a vivência prática do ciclo completo de avaliação de qualidade de software: da definição de requisitos à especificação de métricas, do projeto de casos de teste à interpretação dos resultados. A experiência reforçou a compreensão de que qualidade não é um atributo binário ("tem" ou "não tem"), mas um conjunto multidimensional de características mensuráveis que devem ser monitoradas continuamente ao longo do ciclo de vida do produto.

Por fim, a aplicação das normas ISO 14001 e ISO Guide 82 ampliou a visão do grupo sobre a responsabilidade ambiental e social no desenvolvimento de software, um tema crescentemente relevante em um setor com impacto energético global significativo. O SmartBoard, ao adotar infraestrutura cloud compartilhada e arquitetura modular, já incorpora práticas alinhadas com os princípios de Green Software Engineering, que o grupo pretende aprofundar nas próximas iterações do produto.

O repositório contendo a apresentação completa do sistema está disponível no GitHub:

<https://github.com/ThaynaRFernandes/APS-Sistemas>

6 REFERÊNCIAS BIBLIOGRÁFICAS

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. NBR ISO/IEC 9126: Engenharia de Software — Qualidade de Produto. Rio de Janeiro: ABNT, 2003.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. NBR ISO/IEC 14598: Engenharia de Software — Avaliação de Produto de Software. Rio de Janeiro: ABNT, 2001.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. NBR ISO 14001: Sistemas de Gestão Ambiental — Requisitos com Orientações para Uso. Rio de Janeiro: ABNT, 2015.

INTERNATIONAL ORGANIZATION FOR STANDARDIZATION. ISO Guide 82: Guidelines for Addressing Sustainability in Standards. Genebra: ISO, 2014.

PRESSMAN, Roger S.; MAXIM, Bruce R. Engenharia de Software: Uma Abordagem Profissional. 9. ed. Porto Alegre: AMGH, 2021.

SOMMERVILLE, Ian. Engenharia de Software. 10. ed. São Paulo: Pearson, 2019.

IEEE. IEEE Standard Glossary of Software Engineering Terminology. IEEE Std 610.12-1990. Nova York: IEEE, 1990.

NATIONAL INSTITUTE OF STANDARDS AND TECHNOLOGY (NIST). The Economic Impacts of Inadequate Infrastructure for Software Testing. Gaithersburg: NIST, 2002. (Planning Report 02-3).

OWASP FOUNDATION. OWASP Top Ten 2021. Disponível em: <<https://owasp.org/Top10>>. Acesso em: maio 2025.

SUPABASE. Supabase Documentation. Disponível em: <<https://supabase.com/docs>>. Acesso em: maio 2025.

FLASK. Flask Documentation (3.x). Disponível em: <<https://flask.palletsprojects.com>>. Acesso em: maio 2025.

N8N. n8n Documentation. Disponível em: <<https://docs.n8n.io>>. Acesso em: maio 2025.

GREEN SOFTWARE FOUNDATION. Green Software Practitioner. Disponível em: <<https://learn.greensoftware.foundation>>. Acesso em: maio 2025.

7 FICHA DE ATIVIDADES PRÁTICAS SUPERVISIONADAS

NOME: Thayná Ramos Fernandes

RA: F099EC4

CURSO: Sistemas da Informação

CAMPUS: Paraíso

TURNO: Noturno

CÓDIGO DA ATIVIDADE: 79B3

ANO GRADE: 2026/01

DATA DA ATIVIDADE	DESCRIÇÃO DA ATIVIDADE	TOTAL DE HORAS	ASSINATURA DO ALUNO	HORAS ATRIBUÍDAS (1)
01/05/2026	Pesquisa e desenvolvimento de conceitos gerais	6h	Thayná Ramos Fernandes	
02/05/2026	Pesquisa e desenvolvimento de conceitos gerais	6h	Thayná Ramos Fernandes	
03/05/2026	Desenvolvimento do sistema	6h	Thayná Ramos Fernandes	
08/05/2026	Desenvolvimento do sistema	6h	Thayná Ramos Fernandes	
09/05/2026	Desenvolvimento do sistema	6h	Thayná Ramos Fernandes	
10/05/2026	Documentação do sistema	6h	Thayná Ramos Fernandes	
15/05/2026	Documentação do sistema	6h	Thayná Ramos Fernandes	
16/05/2026	Conclusão e formatação	6h	Thayná Ramos Fernandes	

(1) Horas atribuídas de acordo com o regulamento das Atividades Práticas Supervisionadas do curso.

TOTAL DE HORAS ATRIBUÍDAS: _____

AVALIAÇÃO: _____ Aprovado ou Reprovado

NOTA: _____

DATA: ____/____/____

CARIMBO E ASSINATURA DO COORDENADOR DO CURSO
